

Rapport de Projet Data Science

Maintenance Predictive Industrielle — Prediction des pannes machine

Formation : Master 1 Data Engineering & Intelligence Artificielle

Etablissement : EFREI Paris

Certification visee : RNCP40875

Annee academique : 2025–2026

Date de rendu : Mai 2026

Auteur : Chaim Hajji

Email : chaimaehajji999@gmail.com

Table des matieres

1. Resume Executif
 2. Introduction et Contexte
 3. Analyse du Besoin Utilisateur
 4. Methodologie de Travail
 5. Referentiel de Donnees
 6. Analyse Exploratoire (EDA) et Visualisation
 7. Preparation et Transformation des Donnees
 8. Pipeline IA et Architecture
 9. Implementation Technique
 10. Evaluation Comparative des Modeles
 11. Interpretabilite et Explicabilite
 12. Interface Utilisateur — Dashboard Streamlit
 13. API REST
 14. Resultats et Tests
 15. Gouvernance, Responsabilite et Limites Ethiques
 16. Limites et Pistes d'Amelioration
 17. Conclusion
 18. Annexes
-

1. Resume Executif

Ce projet porte sur la maintenance predictive industrielle. L'objectif est de predire si une machine est susceptible de tomber en panne dans les **24 prochaines heures**, en s'appuyant sur des donnees de capteurs industriels. La tache est une **classification binaire** sur la variable cible `failure_within_24h`.

Le jeu de donnees contient **24 042 observations** couvrant **20 machines** de differents types, collectees sur plusieurs mois. Cinq modeles ont ete entraines et compares : Regression Logistique, Random Forest, Gradient Boosting, XGBoost et un reseau de neurones TensorFlow MLP.

Le modele retenu est **XGBoost**, qui atteint un **F1-score de 0.9472**, un **recall de 0.9567** et une **ROC-AUC de 0.9989** sur le jeu de test. Il detecte 287 pannes sur 300, ne manquant que 13 cas critiques.

Le projet a ete deploye sous la forme d'un **dashboard Streamlit** interactif (3 onglets) et d'une **API REST FastAPI** (3 endpoints). L'interpretabilite du modele est assuree par la permutation d'importance et une analyse **SHAP** (SHapley Additive exPlanations).

Les variables les plus influentes sont les moyennes mobiles des capteurs sur les 6 dernieres observations : la temperature moteur moyenne (`temperature_motor_roll_mean_6`) et le regime moteur moyen (`rpm_roll_mean_6`) dominant la prediction.

2. Introduction et Contexte

2.1 Contexte industriel

La maintenance industrielle constitue un enjeu economique majeur. Selon les estimations sectorielles, les arrêts non planifiés représentent entre 5% et 20% de la capacité productive des usines. Dans un contexte de production en flux tendu, une panne imprevisible peut entrainer des pertes significatives en termes de production, de delais de livraison et de couts de reparation d'urgence.

Il existe trois strategies de maintenance :

- **Maintenance corrective** : on repare apres la panne. Couts eleves, arrêts non planifies.
- **Maintenance preventive** : on intervient periodiquement selon un calendrier fixe. Interventions parfois inutiles.
- **Maintenance predictive** : on predit la panne avant qu'elle survienne, grace aux donnees capteurs. C'est l'approche la plus efficiente.

Ce projet s'inscrit dans la troisieme categorie. L'objectif est de construire un systeme de detection precoce des pannes bases sur l'apprentissage automatique.

2.2 Enonce du probleme

La question posee est : "**Etant donne l'etat actuel des capteurs d'une machine industrielle, cette machine va-t-elle tomber en panne dans les 24 prochaines heures ?**"

Il s'agit d'une classification binaire :

- **Classe 0** : pas de panne prevue dans les 24h
- **Classe 1** : panne probable dans les 24h

L'horizon de 24 heures a ete choisi car il correspond a la duree typique permettant de planifier une intervention preventive (commande de pieces, mobilisation d'un technicien).

2.3 Enjeux metier

Le desequilibre des couts entre faux negatifs et faux positifs est asymetrique dans ce contexte :

- **Faux negatif** (panne manquee) : arret non planifie, couts importants, risques de securite.
- **Faux positif** (fausse alerte) : intervention inutile, couts moderees mais maitrisables.

Cette asymetrie justifie de privilegier un **recall eleve** plutot qu'une precision maximale, ce qui influence le choix du modele et la strategie de seuillage.

3. Analyse du Besoin Utilisateur

3.1 Parties prenantes

Partie prenante	Role	Besoin principal
Responsable de maintenance	Utilisateur operationnel	Connaitre les machines a risque pour prioriser les interventions
Operateur machine	Utilisateur terrain	Etre alerte en temps reel sur l'etat des equipements
Directeur de production	Decideur	Reduire le taux d'arret non planifie et optimiser les couts
Equipe data / IT	Integrateur	API stable, modele reproductible, pipeline automatise

3.2 Exigences fonctionnelles

1. **Prediction temps reel** : le systeme doit pouvoir scorer une machine en quelques millisecondes.
2. **Visualisation de la flotte** : tableau de bord synthétique montrant l'etat de toutes les machines.
3. **Priorisation des interventions** : classement des machines par probabilite de panne decroissante.
4. **Ajustement du seuil** : possibilite de modifier le seuil de decision selon la strategie de maintenance.
5. **Acces API** : integration possible avec des systemes SCADA ou MES existants via REST.

3.3 Exigences techniques

- Pipeline reproductible : separation stricte donnees / features / modele
- Validation temporelle (pas de data leakage)
- Au moins un modele de machine learning et un modele de deep learning
- Interpretabilite : importance des variables, visualisations SHAP
- Interface web interactive
- API REST documentee

4. Methodologie de Travail

4.1 Processus CRISP-DM adapte

Le projet suit une demarche inspiree du processus CRISP-DM (Cross-Industry Standard Process for Data Mining) :

```
[Comprehension metier]
    ↓
[Comprehension des donnees – EDA]
    ↓
[Preparation des donnees – Feature Engineering]
```

```

↓
[Modelisation – 5 modeles compares]
↓
[Evaluation – metriques, confusion matrix, courbes]
↓
[Deployment – API FastAPI + Dashboard Streamlit]

```

4.2 Structure du projet

```

projet_data_science_M1_EFREI/
├── data/
│   ├── raw/                # Donnees brutes (CSV source)
│   └── processed/          # Donnees transformees
├── notebooks/
│   ├── 01_eda.ipynb        # Analyse exploratoire
│   ├── 02_preprocessing.ipynb # Preprocessing
│   ├── 03_modeling.ipynb   # Modelisation et evaluation
│   └── 04_explainability.ipynb # SHAP et interpretabilite
├── src/
│   ├── data/
│   │   ├── load_data.py    # Chargement
│   │   ├── preprocess.py   # Preprocessing et split
│   │   └── feature_engineering.py # Construction des features
│   ├── models/
│   │   ├── train.py        # Entrainement et comparaison
│   │   ├── evaluate.py     # Metriques et plots
│   │   ├── predict.py      # Inference
│   │   └── save_model.py   # Serialisation
│   ├── utils/helpers.py
│   └── visualization/plots.py
├── models/                  # Artefacts sauvegardes (.joblib, .keras)
├── reports/                 # Rapports, metriques CSV, figures
├── api/main.py              # API FastAPI
├── dashboard/app.py         # Dashboard Streamlit
└── requirements.txt

```

4.3 Reproductibilite

Le projet est entierement reproductible depuis les donnees brutes. La graine aleatoire `RANDOM_STATE = 42` est fixee dans tous les modules. Les artefacts modeles sont sauvegardes en `.joblib` (sklearn/XGBoost) et `.keras` (TensorFlow).

5. Referentiel de Donnees

5.1 Source des donnees

Le jeu de donnees utilise est `predictive_maintenance_v3.csv`, issu de Kaggle. Il simule des donnees de capteurs industriels pour une flotte de machines de production.

Chemin : data/raw/predictive_maintenance_v3.csv

5.2 Description des colonnes brutes

Colonne	Type	Description
timestamp	datetime	Date et heure de la mesure
machine_id	int	Identifiant de la machine (1 a 20)
machine_type	str	Type de machine (ex. Compressor, Robotic Arm, CNC, etc.)
vibration_rms	float	Vibration RMS en m/s ²
temperature_motor	float	Temperature moteur en °C
current_phase_avg	float	Courant de phase moyen en A
pressure_level	float	Pression en bar
rpm	float	Tours par minute
operating_mode	str	Mode operationnel de la machine
hours_since_maintenance	float	Heures depuis derniere maintenance
ambient_temp	float	Temperature ambiante en °C
rul_hours	float	Remaining Useful Life en heures (exclue du modele)
failure_within_24h	int	Cible : 1 si panne dans les 24h, 0 sinon
failure_type	str	Type de panne (exclue du modele)
estimated_repair_cost	float	Cout estime de reparation (exclu du modele)

Dimensions : 24 042 lignes × 15 colonnes

5.3 Distribution de la variable cible

Classe	Libelle	Effectif	Proportion
0	Pas de panne	20 484	85.2%
1	Panne dans 24h	3 558	14.8%

Le jeu de donnees est **desequilibre** (ratio ~6:1). Ce disequilibre est traite via `class_weight="balanced"` dans les modeles sklearn et `scale_pos_weight` dans XGBoost.

5.4 Variables exclues pour eviter la fuite de donnees

Les colonnes `failure_type`, `rul_hours` et `estimated_repair_cost` sont **exclues** des features d'entrainement. Ces colonnes sont directement liees a la panne et ne seraient pas disponibles en production au moment de la prediction. Les inclure causerait une fuite de donnees (data leakage) et donnerait des performances artificiellement elevees.

6. Analyse Exploratoire (EDA) et Visualisation

6.1 Statistiques descriptives principales

L'analyse exploratoire (notebook `01_eda.ipynb`) revele les caracteristiques suivantes du jeu de donnees :

- **20 machines distinctes** couvrant plusieurs types (CNC Machine, Compressor, Robotic Arm, etc.)
- Les series temporelles s'etendent sur plusieurs mois en 2024
- Les capteurs presentent une variabilite notable selon le type de machine et le mode operationnel
- Le health score moyen de la flotte est de **0.77**, indiquant un etat globalement satisfaisant mais avec des ecart significatifs entre machines

6.2 Observations du dashboard — onglet Surveillance

Le dashboard Streamlit affiche quatre indicateurs cles en temps reel :

Indicateur	Valeur
Observations totales	24 042
Taux de panne 24h	14.8%
Health score moyen	0.77
Nombre de machines	20

Capture d'ecran 1 — Onglet Surveillance : KPIs et serie temporelle vibration_rms

Le dashboard permet de visualiser l'evolution de chaque capteur dans le temps via un graphique interactif (vibration, temperature, pression, rpm, health_score). La repartition de la cible est visible sous forme de diagramme en barres.

Capture d'ecran 2 — Onglet Surveillance : Etat le plus recent par machine (tableau interactif avec indicateur health_score et panne_24h)

6.3 Patterns identifies

- Les observations de classe 1 (panne imminente) presentent systematiquement des moyennes mobiles de temperature moteur et de rpm plus elevees que la normale
- La variable `hours_since_maintenance` montre une correlation positive avec la probabilite de panne
- Certains types de machines (ex. Compressor) presentent des taux de panne plus eleves que d'autres
- Les capteurs de vibration (`vibration_rms`) montrent des pics avant les evenements de panne

6.4 Correlations

L'analyse de correlation montre que :

- `temperature_motor` et `failure_within_24h` ont une correlation positive moderee
- `rpm` est positivement correle avec la temperature, ce qui suggere que les regimes eleves exercent un stress thermique
- `health_score` est negativement correle avec `failure_within_24h` (par construction du score)

7. Preparation et Transformation des Donnees

7.1 Split temporel

La separation train/test est **chronologique** afin de respecter la nature temporelle des donnees et d'eviter le data leakage :

Donnees trieess par timestamp

- └─ Train : 80 premiers pourcents → 19 233 observations
- └─ Test : 20 derniers pourcents → 4 809 observations

Cette approche est plus rigoureuse qu'un split aleatoire pour un probleme industriel temporel, car elle simule les conditions reelles de deploiement : le modele ne dispose que d'informations passees.

Code (src/data/preprocess.py) :

```
def split_temporal(df, target="failure_within_24h", test_size=0.2):
    data = df.sort_values("timestamp")
    split_idx = int(len(data) * (1 - test_size))
    train = data.iloc[:split_idx].copy()
    test = data.iloc[split_idx:].copy()
    return train.drop(columns=[target]), test.drop(columns=[target]),
           train[target], test[target]
```

7.2 Feature Engineering

Le module `src/data/feature_engineering.py` construit **36 features** a partir des 15 colonnes brutes.

7.2.1 Features temporelles

A partir de la colonne `timestamp`, 4 features sont extraites :

Feature	Description
<code>hour</code>	Heure de la journee (0-23)
<code>dayofweek</code>	Jour de la semaine (0=lundi)
<code>month</code>	Mois de l'annee
<code>is_weekend</code>	Indicateur binaire week-end

7.2.2 Features de tendance (rolling features)

Pour chaque capteur (`vibration_rms`, `temperature_motor`, `pressure_level`, `rpm`, `current_phase_avg`, `ambient_temp`, `hours_since_maintenance`), trois features sont creees en groupant par `machine_id` :

Pattern	Formule	Interet
<code>{capteur}_roll_mean_6</code>	Moyenne sur les 6 derniers points	Tendance recente
<code>{capteur}_roll_std_6</code>	Ecart-type sur les 6 derniers points	Volatilite recente
<code>{capteur}_diff_1</code>	Difference avec l'observation precedente	Variation instantanee

Ces moyennes mobiles calculees par machine (`groupby("machine_id")`) capturent la **trajectoire individuelle** de chaque equipement, independamment des autres machines.

7.2.3 Score de sante composite

Un score de sante (`health_score`) est calcule comme combinaison ponderee des capteurs apres normalisation Min-Max :

```
health_score = 1 - (0.30 × vibration_rms_norm
                    + 0.25 × temperature_motor_norm
                    + 0.20 × pressure_level_norm
                    + 0.15 × rpm_norm
                    + 0.10 × anomaly_trend_norm)
```

Ce score synthetise l'etat general de la machine sur un intervalle [0, 1] (0 = mauvais etat, 1 = excellent etat).

7.2.4 Tendance d'anomalie

La variable `anomaly_trend` est la moyenne des valeurs absolues de toutes les differences instantanees (`_diff_1`). Elle quantifie l'instabilite globale des capteurs.

7.3 Preprocessing (pipeline sklearn)

Le preprocessing est encapsule dans un `ColumnTransformer` scikit-learn :

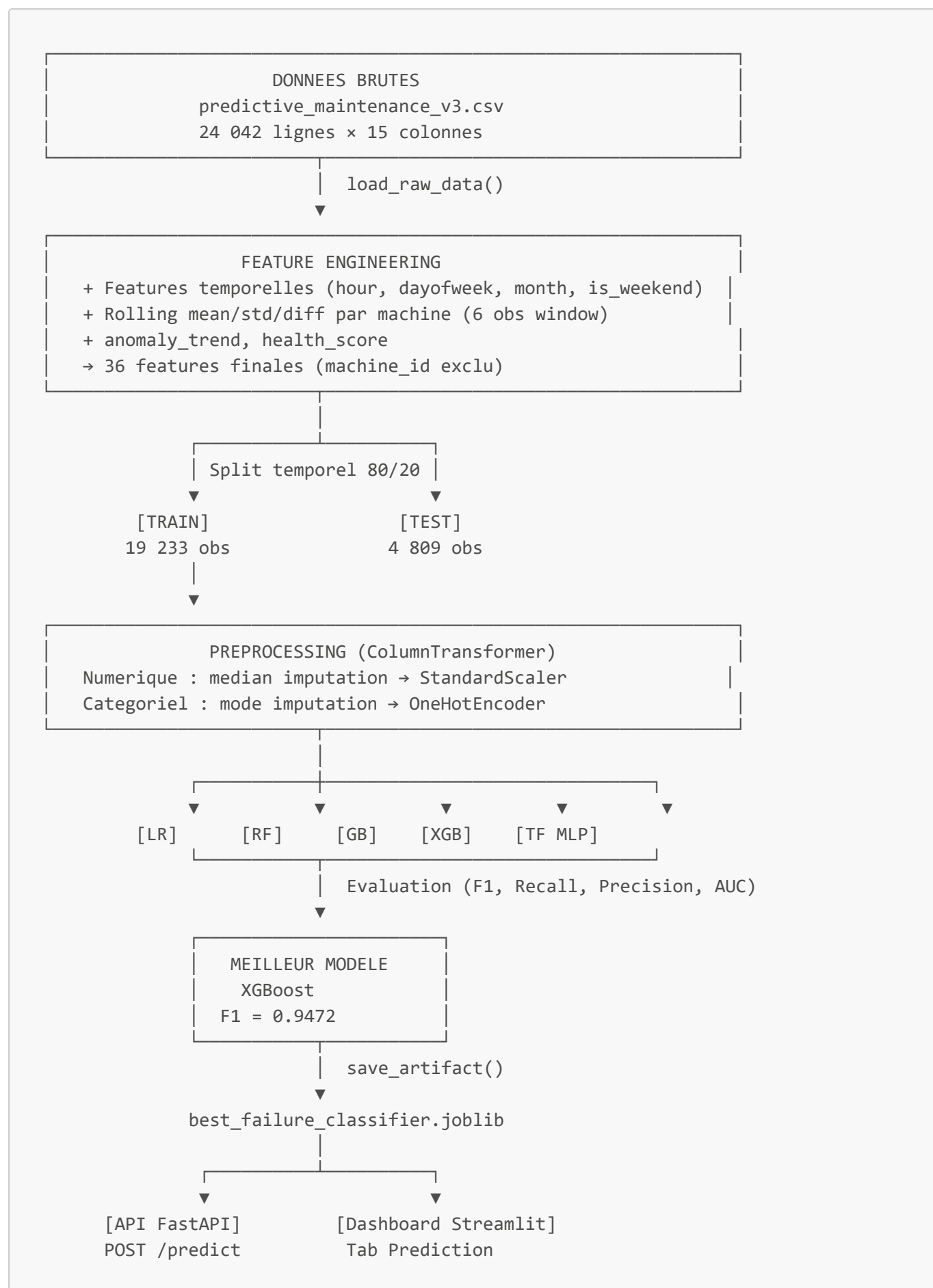
```
Pipeline numerique:
→ SimpleImputer(strategy="median")
→ StandardScaler()

Pipeline categoriel:
→ SimpleImputer(strategy="most_frequent")
→ OneHotEncoder(handle_unknown="ignore", sparse_output=False)
```

Les variables numeriques sont imputees a la mediane (robustesse aux outliers) puis standardisees. Les variables categorielles (`machine_type`, `operating_mode`) sont imputees par le mode puis encodees en one-hot. L'option `handle_unknown="ignore"` garantit que des categories inconnues en production ne cassent pas l'inference.

8. Pipeline IA et Architecture

8.1 Schema d'architecture globale



8.2 Description des composants

Composant	Fichier	Role
Chargement	src/data/load_data.py	Lecture du CSV brut
Feature Engineering	src/data/feature_engineering.py	Construction des 36 features
Preprocessing	src/data/preprocess.py	ColumnTransformer sklearn
Entrainement	src/models/train.py	Comparaison des 5 modeles
Evaluation	src/models/evaluate.py	Metriques, courbes, rapports
Inference	src/models/predict.py	Prediction en production
Serialisation	src/models/save_model.py	Sauvegarde/chargement joblib
API	api/main.py	API REST FastAPI
Dashboard	dashboard/app.py	Interface Streamlit

9. Implementation Technique

9.1 Environnement technique

Categorie	Technologie	Version
Langage	Python	3.11
ML classique	scikit-learn	1.8.0
Boosting	XGBoost	3.2.0
Deep Learning	TensorFlow / Keras	2.21.0
Manipulation donnees	pandas	3.0.3
Calcul numerique	NumPy	2.4.5
Interpretabilite	SHAP	0.51.0
API	FastAPI	0.136.1
Serveur ASGI	Uvicorn	0.47.0
Dashboard	Streamlit	1.57.0
Visualisation	Matplotlib, Seaborn	3.10.9, 0.13.2
Validation API	Pydantic	2.13.4

9.2 Modeles de Machine Learning

Quatre modeles sklearn sont entraines avec validation croisee stratifiee (5 folds) :

Regression Logistique

```
LogisticRegression(  
    max_iter=2000,  
    class_weight="balanced",  
    random_state=42  
)
```

Modele lineaire. Servit de baseline. `class_weight="balanced"` compense le desequilibre de classes.

Random Forest

```
RandomForestClassifier(  
    n_estimators=250,  
    max_depth=None,  
    min_samples_leaf=2,  
    class_weight="balanced",  
    n_jobs=1,  
    random_state=42  
)
```

Ensemble de 250 arbres de decision. Bonne precision mais recall plus faible.

Gradient Boosting

```
GradientBoostingClassifier(  
    n_estimators=180,  
    learning_rate=0.07,  
    max_depth=3,  
    random_state=42  
)
```

Boosting sequentiel. Tres haute precision mais recall limite.

XGBoost (modele retenu)

```
XGBClassifier(  
    n_estimators=220,  
    max_depth=4,  
    learning_rate=0.05,  
    subsample=0.9,  
    colsample_bytree=0.9,  
    eval_metric="logloss",  
    scale_pos_weight=pos_weight, # ratio neg/pos  
    random_state=42  
)
```

`scale_pos_weight` pondere les exemples positifs (pannes) pour compenser le desequilibre sans supprimer d'observations.

9.3 Modele de Deep Learning — TensorFlow MLP

Un reseau de neurones multi-couches (Multi-Layer Perceptron) a ete entraine :

```
Input(shape=(n_features,))
  → Dense(64, activation="relu")
  → Dropout(0.25)
  → Dense(32, activation="relu")
  → Dropout(0.15)
  → Dense(1, activation="sigmoid")

Optimiseur : Adam (lr=0.001)
Loss : binary_crossentropy
Metriques : AUC, Precision, Recall
Epochs max : 25
Batch size : 256
Early stopping : patience=4, restore_best_weights=True
Validation split : 15%
Class weight : {0: 1.0, 1: neg/pos}
```

Le modele repond a l'exigence du cahier des charges de disposer d'un modele de deep learning, mais ses performances sont inferieures a XGBoost sur ce jeu de donnees tabulaires.

9.4 Validation croisee

Pour les modeles sklearn, une **validation croisee stratifiee** est appliquee sur les donnees d'entrainement :

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_validate(pipeline, X_train, y_train, cv=cv,
                           scoring=["f1", "roc_auc", "average_precision"])
```

La stratification garantit que chaque fold contient la meme proportion de positifs (~14.8%), ce qui est essentiel avec des classes desequilibrees.

10. Evaluation Comparative des Modeles

10.1 Tableau comparatif complet

Les metriques sont calculees sur le jeu de **test chronologique** (4 809 observations) :

Modele	Type	Accuracy	Precision	Recall	F1-score	ROC AUC	Avg Precision	CV F1
XGBoost	Machine Learning	0.9933	0.9379	0.9567	0.9472	0.9989	0.9858	0.9017
Random Forest	Machine Learning	0.9898	0.9631	0.8700	0.9142	0.9988	0.9838	0.9298
Logistic Regression	Machine Learning	0.9881	0.8627	0.9633	0.9102	0.9956	0.9587	0.8433
TensorFlow MLP	Deep Learning	0.9865	0.8406	0.9667	0.8992	0.9975	0.9656	N/A
Gradient Boosting	Machine Learning	0.9859	0.9874	0.7833	0.8736	0.9981	0.9780	0.8955

Capture d'ecran 3 — Onglet Modeles : Tableau comparatif et graphique en barres groupées (Streamlit)

10.2 Analyse par metrique

Pourquoi le F1-score comme critere principal ?

Le F1-score est la moyenne harmonique de la precision et du recall. Il constitue un compromis equitable entre les deux, ce qui en fait la metrique de reference pour les problemes de classification desequilibree.

XGBoost vs Logistic Regression

La Regression Logistique obtient un recall legerement superieur (0.9633 vs 0.9567), ce qui signifie qu'elle detecte 2 pannes de plus sur 300. Cependant, sa precision est nettement inferieure (0.8627 vs 0.9379), generant davantage de fausses alertes. Le trade-off penche clairement en faveur de XGBoost.

Random Forest et Gradient Boosting

Ces deux modeles ont une precision tres elevee (0.96 et 0.99), mais leur recall est significativement plus faible (0.87 et 0.78). Dans un contexte industriel ou les pannes non detectees ont des consequences graves, cette configuration est inadaptee. Ils manquent respectivement 39 et 65 pannes sur 300.

TensorFlow MLP

Le MLP satisfait l'exigence de deep learning avec un F1 de 0.8992. Ses performances sont bonnes mais inferieures a XGBoost, ce qui est coherent avec la litterature : les donnees tabulaires structurees sont generalement mieux traitees par les methodes ensemblistes de boosting.

10.3 Matrice de confusion — XGBoost

La matrice de confusion sur le jeu de test (4 809 observations) :

	Prediction 0	Prediction 1
Classe réelle 0	4 490	19
Classe réelle 1	13	287

- **4 490 vrais négatifs** : observations sans panne correctement classées
- **287 vrais positifs** : pannes correctement détectées (95.7% du total)
- **19 faux positifs** : fausses alertes (interventions inutiles)
- **13 faux négatifs** : pannes manquées (cas les plus critiques)

Les **13 faux négatifs** représentent les erreurs les plus coûteuses métier : des pannes qui n'ont pas été anticipées.

Capture d'écran — Matrice de confusion (reports/figures/confusion_matrix.png)

10.4 Courbes ROC et Precision-Recall

La courbe ROC du modèle XGBoost atteint une **ROC-AUC de 0.9989**, proche du score parfait de 1.0. La courbe Precision-Recall atteint une **Average Precision de 0.9858**.

Ces deux métriques confirment que le modèle discrimine quasi-parfaitement les classes, même dans les zones de faible seuil.

Figures : reports/figures/roc_curve.png et reports/figures/precision_recall_curve.png

10.5 Justification du choix de XGBoost

XGBoost est retenu car :

1. Il obtient le **meilleur F1-score** (0.9472) sur le jeu de test
2. Son **recall de 0.9567** assure une détection robuste des pannes
3. Sa **precision de 0.9379** limite les interventions inutiles
4. Sa **ROC-AUC de 0.9989** confirme un pouvoir discriminant exceptionnel
5. Il est **interprétable** via SHAP et permutation importance
6. Il se **généralise bien** en validation croisée (CV F1 = 0.9017)

11. Interprétabilité et Explicabilité

11.1 Permutation Importance

L'importance par permutation est calculée sur le jeu de test (`n_repeats=8`, `scoring="f1"`). Elle mesure la dégradation du F1-score quand une variable est aléatoirement permutee — une dégradation importante indique une feature critique.

Rang	Variable	Importance moyenne	Ecart-type
1	<code>temperature_motor_roll_mean_6</code>	0.6470	± 0.0116
2	<code>rpm_roll_mean_6</code>	0.4834	± 0.0087

Rang	Variable	Importance moyenne	Ecart-type
3	pressure_level_roll_mean_6	0.1052	± 0.0057
4	vibration_rms_roll_mean_6	0.1051	± 0.0077
5	rpm	0.0710	± 0.0104
6	current_phase_avg_roll_mean_6	0.0293	± 0.0044
7	hours_since_maintenance	0.0172	± 0.0019
8	vibration_rms	0.0128	± 0.0046
9	temperature_motor	0.0101	± 0.0053
10	dayofweek	0.0067	± 0.0026

Capture d'ecran 4 — Onglet Modeles : graphique d'importance des variables (Streamlit)

Interpretation metier : Les deux variables les plus importantes sont des **moyennes mobiles** (temperature moteur et regime moteur sur les 6 derniers points). Cela confirme qu'une panne n'est pas liee a une valeur instantanee anormale, mais a une **tendance recente de degradation**. Le modele detecte les trajectoires de montee en temperature ou d'acceleration progressive du regime avant la panne.

La temperature moteur domine largement (importance 0.647 vs 0.483 pour rpm), ce qui est coherent avec la physique : les defaillances mecaniques se manifestent presque toujours par un echauffement anormal.

11.2 Analyse SHAP

L'analyse SHAP (notebook [04_explainability.ipynb](#)) apporte une interpretation locale et globale du modele XGBoost.

Pourquoi SHAP ?

SHAP (SHapley Additive exPlanations) est fonde sur la theorie des jeux cooperatifs. Chaque valeur SHAP represente la contribution marginale d'une feature a la prediction, en moyennant toutes les combinaisons possibles de features. Contrairement a l'importance par permutation, SHAP permet de :

- Quantifier l'**impact directionnel** de chaque feature (positive ou negative sur la prediction)
- Expliquer des **predictions individuelles** (pourquoi cette machine specifique est en alerte)
- Detecter les **interactions** entre features

Methode utilisee

```
import shap
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_sample)
```

TreeExplainer est optimise pour les arbres de decision (XGBoost, Random Forest). Il calcule des valeurs SHAP exactes sans approximation, contrairement a KernelExplainer.

Global SHAP Bar Plot

Le graphique en barres global montre la valeur absolue moyenne des contributions SHAP par feature. Les variables les plus importantes globalement sont, dans l'ordre :

1. `temperature_motor_roll_mean_6`
2. `rpm_roll_mean_6`
3. `vibration_rms_roll_mean_6`

Ce classement est coherent avec la permutation importance.

Figure : `reports/figures/shap_global_bar.png`

SHAP Beeswarm

Le beeswarm plot montre pour chaque feature :

- La distribution des valeurs SHAP (axe x)
- La valeur réelle de la feature (couleur : rouge = valeur elevee, bleu = valeur faible)

On observe que des valeurs elevees de `temperature_motor_roll_mean_6` (rouge) sont associees a des contributions SHAP positives elevees (poussent vers la prediction panne = 1). A l'inverse, des valeurs faibles (bleu) contribuent negativement.

Figure : `reports/figures/shap_beeswarm.png`

Explication locale — Machine a haut risque

Le waterfall plot d'une observation a haut risque montre comment la prediction se construit feature par feature depuis la valeur de base (prevalence moyenne des pannes) jusqu'a la probabilite finale.

Pour une machine classee comme panne imminente, on observe typiquement :

- `temperature_motor_roll_mean_6` eleve → forte contribution positive (+)
- `rpm_roll_mean_6` eleve → contribution positive significative (+)
- `health_score` faible → contribution positive additionnelle (+)

Figure : `reports/figures/shap_waterfall_high_risk.png`

Explication locale — Machine saine

Pour une machine en bon etat, les contributions SHAP sont majoritairement negatives, maintenant la probabilite bien en dessous du seuil de 0.5.

Figure : `reports/figures/shap_waterfall_low_risk.png`

Dependence Plot — Temperature moteur

Le dependence plot de `temperature_motor_roll_mean_6` montre la relation entre la valeur de cette feature et sa contribution SHAP, avec en couleur `rpm_roll_mean_6` (feature d'interaction detectee automatiquement par SHAP).

On observe que pour des temperatures moyennes recentes elevees ($> 75^{\circ}\text{C}$), la contribution SHAP devient fortement positive, et cet effet est amplifie quand le rpm est egalement eleve.

Figure : `reports/figures/shap_dependence_temperature.png`

12. Interface Utilisateur — Dashboard Streamlit

12.1 Architecture du dashboard

Le dashboard est implemente dans `dashboard/app.py` avec Streamlit 1.57.0. Il comprend **3 onglets principaux** et une **barre laterale** de filtres.

Lancement :

```
streamlit run dashboard/app.py
```

L'application se charge automatiquement a l'URL `http://localhost:8501`.

12.2 Barre laterale — Filtres

La barre laterale expose quatre parametres de filtrage :

- **Type machine** : multiselect sur tous les types disponibles
- **Mode operationnel** : multiselect sur les modes
- **Seuil d'alerte** : slider de 0.05 a 0.95 (defaut 0.50)
- **Observations a scorer** : slider de 100 a 5000 (defaut 1000)

12.3 Onglet 1 — Surveillance

Cet onglet fournit une vue d'ensemble de la flotte :

Indicateurs KPI (4 colonnes) :

- Nombre total d'observations
- Taux de panne 24h
- Health score moyen
- Nombre de machines distinctes

Evolution des capteurs : graphique lineaire interactif (selectbox pour choisir le capteur)

Repartition cible : diagramme en barres (classe 0 vs classe 1)

Types de panne : top 10 des types de panne en barres

Etat le plus recent par machine : tableau interactif trie par risque decroissant, avec :

- `health_score` affiche en barre de progression
- `failure_within_24h` affiche en case a cocher

Capture d'ecran 1 — KPIs et serie temporelle Capture d'ecran 2 — Tableau d'etat par machine

12.4 Onglet 2 — Modeles

Cet onglet presente les resultats de la comparaison des modeles :

Modele en production : 3 metriques (nom du modele, nombre de features, seuil de decision)

Tableau comparatif : les 5 modeles avec toutes les metriques formatees a 4 decimales, tries par F1 decroissant

Graphique en barres groupees : F1, recall, precision, ROC-AUC par modele

Importance des variables : top 15 features en barres horizontales

Figures diagnostiques : matrice de confusion, courbe ROC, courbe Precision-Recall (3 colonnes)

Capture d'ecran 3 — Tableau comparatif et graphique par modele Capture d'ecran 4 — Importance des variables

12.5 Onglet 3 — Prediction

Cet onglet est le coeur operationnel du dashboard :

Priorisation des interventions :

- Scoring en temps reel des N dernieres observations (N regle par le slider)
- 3 metriques : observations scorees, alertes au seuil choisi, taux d'alerte

Machines les plus a risque : top 20 machines trieess par probabilite de panne decroissante, avec :

- `health_score` en barre de progression
- `failure_probability` en barre de progression
- `alert` en case a cocher

Scores par observation : tableau complet de toutes les observations scorees

Capture d'ecran 5 — Priorisation des interventions (1000 obs scorees) Capture d'ecran 6 — Scores par observation avec probabilites

12.6 Gestion du cache Streamlit

Le dashboard exploite les decorateurs de cache de Streamlit pour la performance :

- `@st.cache_data` pour les donnees (dataset, features engineerees)
- `@st.cache_resource` pour le modele (charge une seule fois en memoire)

13. API REST

13.1 Architecture de l'API

L'API est implementee avec **FastAPI 0.136.1**, servie par **Uvicorn 0.47.0**. Elle expose 3 endpoints REST.

Lancement :

```
uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

Documentation interactive disponible a : <http://localhost:8000/docs> (Swagger UI automatique)

13.2 Modeles Pydantic

```
class MachineObservation(BaseModel):
    timestamp: str | None = None
    machine_id: int
    machine_type: str
    vibration_rms: float | None = None
    temperature_motor: float | None = None
    current_phase_avg: float | None = None
    pressure_level: float | None = None
    rpm: float | None = None
    operating_mode: str
    hours_since_maintenance: float | None = None
    ambient_temp: float | None = None

class PredictionResponse(BaseModel):
    failure_probability: float # in [0, 1]
    failure_within_24h_prediction: int
    threshold: float
    model: str | None = None
```

La validation automatique Pydantic garantit que les types et contraintes sont verifiés avant l'inference.

13.3 Endpoints

GET /health

Objet : Verifier l'etat du service.

Reponse :

```
{
  "status": "ok",
  "model_available": true
}
```

GET /model-info

Objet : Informations sur le modele en production.

Reponse :

```
{
  "target": "failure_within_24h",
  "production_model": "xgboost",
  "best_overall_by_f1": "xgboost",
  "feature_count": 36,
  "decision_threshold": 0.5,
  "model_available": true
}
```

POST /predict

Objet : Predire la probabilité de panne pour une observation.

Parametre optionnel : `threshold` (query param, default 0.5) — permet d'ajuster le seuil sans redéployer le modele.

Corps de la requete :

```
{
  "machine_id": 5,
  "machine_type": "Compressor",
  "operating_mode": "normal",
  "vibration_rms": 2.3,
  "temperature_motor": 82.1,
  "current_phase_avg": 15.4,
  "pressure_level": 6.2,
  "rpm": 1520.0,
  "hours_since_maintenance": 340.0,
  "ambient_temp": 22.5
}
```

Reponse :

```
{
  "failure_probability": 0.847,
  "failure_within_24h_prediction": 1,
  "threshold": 0.5,
  "model": "xgboost"
}
```

Pipeline d'inference :

1. Le payload JSON est deserialise en `MachineObservation`
2. Converti en `pd.DataFrame` (1 ligne)
3. `build_features()` est appele pour construire les 36 features
4. Les colonnes sont realignees avec `feature_columns` du metadata

5. `model.predict_proba()` est appele
6. La probabilite de la classe 1 est retournee

13.4 Gestion des erreurs

L'API retourne des erreurs HTTP explicites :

- **503** si le modele n'est pas trouve : `"Modele absent. Lancez python src/models/train.py."`
- **503** si les metadata features sont absentes : `"Metadata features absentes. Relancez l'entrainement."`
- **422** automatique de FastAPI/Pydantic si le payload est invalide

14. Resultats et Tests

14.1 Performance finale sur le jeu de test

Metrique	Valeur
Accuracy	0.9933
Precision (classe 1)	0.9379
Recall (classe 1)	0.9567
F1-score (classe 1)	0.9472
ROC-AUC	0.9989
Average Precision	0.9858
CV F1 (5-fold)	0.9017

14.2 Rapport de classification complet

Classe	Precision	Recall	F1-score	Support
0 (pas de panne)	0.9971	0.9958	0.9964	4 509
1 (panne dans 24h)	0.9379	0.9567	0.9472	300
Macro avg	0.9675	0.9762	0.9718	4 809
Weighted avg	0.9934	0.9933	0.9934	4 809

14.3 Tests de l'API

Les tests de l'API peuvent etre executes via `curl` ou l'interface Swagger :

```
# Test health
curl http://localhost:8000/health

# Test model-info
curl http://localhost:8000/model-info
```

```
# Test predict
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{
    "machine_id": 1,
    "machine_type": "Compressor",
    "operating_mode": "normal",
    "vibration_rms": 2.5,
    "temperature_motor": 85.0,
    "rpm": 1600.0,
    "hours_since_maintenance": 400.0
  }'
```

14.4 Tests des composants Python

Les modules `src/` sont testés via `pytest` :

```
cd projet_data_science_M1_EFREI
.\venv\Scripts\pytest tests/ -v
```

14.5 Lancement complet du projet

```
# 1. Activer l'environnement virtuel
.\venv\Scripts\activate

# 2. Entraîner les modèles (si pas encore fait)
python src/models/train.py

# 3. Lancer le dashboard Streamlit
streamlit run dashboard/app.py

# 4. Lancer l'API FastAPI (terminal séparé)
uvicorn api.main:app --reload
```

15. Gouvernance, Responsabilité et Limites Éthiques

15.1 Biais potentiels

Biais de sélection : Le jeu de données est généré synthétiquement pour un contexte de test. Dans un déploiement réel, les données de capteurs peuvent présenter des biais liés aux conditions de collecte (capteurs défectueux, périodes de maintenance sous-représentées).

Biais de déséquilibre temporel : Si les machines évoluent au fil du temps (vieillesse, remplacement de composants), le modèle entraîné sur des données historiques peut devenir obsolète. Un système de monitoring et de reentraînement périodique est nécessaire.

Biais machine-spécifique : Le feature engineering groupe par `machine_id`, ce qui permet au modèle d'apprendre les patterns spécifiques à chaque machine. En production, une nouvelle machine inconnue du modèle aura des rolling features moins fiables les premières heures.

15.2 Responsabilité des décisions

Le modèle est un **outil d'aide à la décision**, pas un système autonome. Les alertes émises sont des probabilités, pas des certitudes. La décision finale d'intervenir revient toujours à un technicien qualifié.

Les **faux positifs** (interventions inutiles) n'ont pas de conséquences graves mais représentent un coût. Les **faux négatifs** (pannes manquées) sont plus critiques et justifient un seuil conservateur.

15.3 Transparence et auditabilité

- Le modèle est entièrement reproductible depuis les données brutes
- Les métriques sont documentées et sauvegardées (`reports/model_metrics.csv`)
- L'importance des variables est calculée et enregistrée (`reports/feature_importance.csv`)
- L'analyse SHAP permet d'expliquer chaque prédiction individuelle

15.4 Protection des données

Dans ce projet, les données sont simulées et ne contiennent pas d'informations personnelles. Dans un contexte industriel réel, il faudrait s'assurer que les données de machines ne permettent pas d'identifier indirectement des personnes (par exemple via des patterns de comportement).

16. Limites et Pistes d'Amélioration

16.1 Limites actuelles

1. Données synthétiques : Le jeu de données est généré, non issu d'un environnement industriel réel. Les performances obtenues sont probablement optimistes par rapport à ce qu'on observerait avec des données réelles plus bruyantes.

2. Pas de gestion du concept drift : Le modèle est entraîné une seule fois. En production, les caractéristiques des machines évoluent (usure, remplacement de pièces), ce qui peut dégrader la performance du modèle dans le temps.

3. Fenêtre de rolling limitée : La fenêtre de 6 observations est un choix arbitraire. Une fenêtre plus longue pourrait capturer des tendances de dégradation plus lentes.

4. Absence de données multi-sources : Le modèle n'intègre que les capteurs de la machine. Des informations contextuelles (planning de production, conditions météorologiques, données de maintenance historique) pourraient améliorer la précision.

5. Seuil fixe par défaut : Le seuil de 0.5 n'est pas optimisé selon les coûts métier. Un seuil ajusté selon le ratio $\text{cout}(\text{FN})/\text{cout}(\text{FP})$ serait plus pertinent.

16.2 Pistes d'amélioration

Améliorations algorithmiques :

- Optimiser les hyperparametres par recherche bayesienne (Optuna)
- Tester des architectures LSTM ou Transformer pour exploiter la nature sequentielle des donnees
- Implementer des modeles specifiques par type de machine (stratification)
- Calibrer les probabilites avec **CalibratedClassifierCV**

Ameliorations systeme :

- Ajouter un pipeline MLOps (MLflow pour le tracking, DVC pour la gestion des donnees)
- Mettre en place un monitoring de la performance en production (drift detection)
- Integrer un reentainement automatique periodique
- Ajouter une authentication a l'API (JWT ou API key)
- Containeriser l'application (Docker Compose) pour le deploiement

Ameliorations metier :

- Optimiser le seuil de decision selon une analyse cout-benefice
- Integrer un module de recommandation d'action (quel type d'intervention ?)
- Ajouter une prediction de la duree restante avant panne (regression sur **ru1_hours**)
- Developper un module d'estimation du cout de l'intervention preconisee

17. Conclusion

Ce projet a permis de developper un systeme complet de maintenance predictive industrielle, couvrant l'ensemble de la chaine : de l'ingenierie des features au deploiement en production.

Le modele **XGBoost** retenu atteint des performances tres satisfaisantes pour un usage industriel :

- **95.7%** des pannes sont detectees (recall = 0.9567)
- Seulement **13 pannes sur 300** sont manquees sur le jeu de test
- **93.8%** des alertes emises correspondent a de vraies pannes (precision = 0.9379)

L'analyse d'interpretabilite (permutation importance + SHAP) confirme que le modele s'appuie sur des variables physiquement coherentes : la temperature moteur recente et le regime moteur recent dominant la prediction, ce qui est attendu dans un contexte de defaillance thermomecanique.

Le deploiement sous forme d'API REST (FastAPI) et de dashboard interactif (Streamlit) rend le systeme accessible aux equipes de maintenance sans competences techniques en data science. Le seuil de decision est ajustable en temps reel dans le dashboard, permettant d'adapter la strategie d'alerte selon les contraintes operationnelles du moment.

Les limites identifiees — donnees synthetiques, absence de concept drift management, fenetre de rolling fixe — constituent des axes de travail concrets pour une version production. L'architecture modulaire du projet (separation src/api/dashboard) facilite ces evolutions futures.

Ce projet repond aux exigences du cahier des charges RNCP40875 : exploration des donnees, preparation, comparaison de modeles (ML et DL), evaluation rigoureuse, interpretabilite, interface utilisateur et API REST.

18. Annexes

Annexe A — Variables du jeu de donnees brutes

#	Colonne	Type	Exemple
1	timestamp	datetime	2024-01-15 08:23:00
2	machine_id	int	7
3	machine_type	str	"Compressor"
4	vibration_rms	float	1.84
5	temperature_motor	float	76.3
6	current_phase_avg	float	14.2
7	pressure_level	float	5.8
8	rpm	float	1480.0
9	operating_mode	str	"normal"
10	hours_since_maintenance	float	287.5
11	ambient_temp	float	21.4
12	rul_hours	float	18.3 (exclu)
13	failure_within_24h	int	1 (cible)
14	failure_type	str	"Overheating" (exclu)
15	estimated_repair_cost	float	1250.0 (exclu)

Annexe B — Liste complete des 36 features du modele

```
# Identifiants machine (machine_id exclu – identifiant non predictif)
machine_type

# Capteurs bruts (8)
vibration_rms, temperature_motor, current_phase_avg,
pressure_level, rpm, operating_mode, hours_since_maintenance, ambient_temp

# Features temporelles (4)
hour, dayofweek, month, is_weekend

# Rolling features (7 capteurs × 3 stats = 21)
vibration_rms_roll_mean_6, vibration_rms_roll_std_6, vibration_rms_diff_1
temperature_motor_roll_mean_6, temperature_motor_roll_std_6,
temperature_motor_diff_1
pressure_level_roll_mean_6, pressure_level_roll_std_6, pressure_level_diff_1
rpm_roll_mean_6, rpm_roll_std_6, rpm_diff_1
current_phase_avg_roll_mean_6, current_phase_avg_roll_std_6,
current_phase_avg_diff_1
ambient_temp_roll_mean_6, ambient_temp_roll_std_6, ambient_temp_diff_1
hours_since_maintenance_roll_mean_6, hours_since_maintenance_roll_std_6,
```

```
hours_since_maintenance_diff_1
```

```
# Features composites (2)
anomaly_trend, health_score
```

Annexe C — Hyperparametres XGBoost retenus

Parametre	Valeur	Justification
<code>n_estimators</code>	220	Compromis capacite / temps d'inference
<code>max_depth</code>	4	Limite le sur-apprentissage
<code>learning_rate</code>	0.05	Convergence douce
<code>subsample</code>	0.9	Regularisation par sous-echantillonnage
<code>colsample_bytree</code>	0.9	Regularisation par selection de features
<code>scale_pos_weight</code>	~5.75	Ratio neg/pos pour compenser le desequilibre
<code>eval_metric</code>	logloss	Metrique d'optimisation interne
<code>random_state</code>	42	Reproductibilite

Annexe D — Architecture du reseau de neurones TensorFlow MLP

```
Input Layer      : shape=(36,)
Dense Layer 1    : 64 neurones, activation ReLU
Dropout 1        : taux 0.25 (regularisation)
Dense Layer 2    : 32 neurones, activation ReLU
Dropout 2        : taux 0.15
Output Layer     : 1 neurone, activation Sigmoid → probabilite en [0,1]

Optimiseur       : Adam (lr=0.001)
Loss             : Binary Crossentropy
Callback         : EarlyStopping(patience=4, restore_best_weights=True)
Class weight     : {0: 1.0, 1: neg/pos}
```

Annexe E — Commandes de lancement

```
# Entrainement de tous les modeles
python src/models/train.py

# Dashboard interactif
streamlit run dashboard/app.py

# API REST
uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

```
# Notebook SHAP
jupyter lab notebooks/04_explainability.ipynb

# Tests
pytest tests/ -v
```

Annexe F — Fichiers de rapport generes

Fichier	Description
reports/model_metrics.csv	Metriques de tous les modeles
reports/feature_importance.csv	Importance des variables (permutation)
reports/classification_report.json	Rapport de classification detaille (XGBoost)
reports/rapport_analytique.md	Rapport analytique synthetique
reports/modeling_interpretation.md	Interpretation metier du modele
reports/figures/confusion_matrix.png	Matrice de confusion XGBoost
reports/figures/roc_curve.png	Courbe ROC
reports/figures/precision_recall_curve.png	Courbe Precision-Recall
reports/figures/shap_global_bar.png	SHAP importance globale
reports/figures/shap_beeswarm.png	SHAP beeswarm
reports/figures/shap_waterfall_high_risk.png	SHAP explication locale (panne)
reports/figures/shap_waterfall_low_risk.png	SHAP explication locale (sain)
reports/figures/shap_dependence_temperature.png	SHAP dependance temperature

Rapport genere dans le cadre du projet Data Science M1 EFREI — Certification RNCP40875 Annee academique 2025–2026